PROYECTO FINAL

RESACA

MEMORIA

Mario Villanueva Torres

Programación de Videojuegos

Máster de Desarrollo de Videojuegos (CEI.)

CEI.

ÍNDICE

INTRODUCCIÓN 2
MENÚ PRINCIPAL 3
CINEMÁTICA 5
PARTIDA 6
MINIJUEGOS 9
"TIRO AL BLANCO" 9
"VIENTO Y MADERA" 12
SISTEMA DE GUARDADO 14
CONFLICTOS 15

INTRODUCCIÓN

RESACA es un proyecto elaborado en el marco de Trabajo final para el módulo de Programación de Videojuegos del máster de Desarrollo de Videojuegos del Centro de Estudios de Innovación (CEI.) de Madrid.

Debido a limitaciones temporales y logísticas, actualmente RESACA encaja más en el concepto de una DEMO jugable de un potencial desarrollo futuro más completo. Es por ello, y para favorecer la rejugabilidad, que el juego presenta dos funcionalidades diferenciadas:

- Un "Modo historia", que representa una partida al uso, con una línea temática coherente, que ofrece al jugador pequeños desafíos para avanzar en una trama argumental ficticia.
- Una selección de minijuegos, que como su propio nombre indica, permite elegir entre una breve lista de juegos (desbloqueados previamente en el "Modo historia") y acceder a ellos libremente con determinadas características a mayores.

RESACA cuenta con más de 40 *scripts* y alrededor de 4000 líneas de código que dan cuerpo y forma al proyecto. También cuenta con más de 100 *assets* y paquetes externos, todos de uso libre y gratuito, referenciados adecuadamente en un panel de "Agradecimientos" en el menú principal del juego.

Es de importancia destacar que el uso de Inteligencia Artificial[1] fue ampliamente evitado durante el proceso creativo y de desarrollo de este proyecto. Sin embargo, la IA sí fue utilizada como recurso paliativo y de corrección de errores.

En el desarrollo de esta memoria procederé a desgranar por escenas las distintas partes que componen RESACA en su conjunto, destacando aquellas características que, a mi parecer, requieren de una mayor atención por su complejidad (relativa al punto de mi formación) o importancia en el resultado final.

[1] En este contexto, el concepto de IA se refiere al uso de los popularmente conocidos como chatbots o asistentes online (ChatGPT, Grok, Copilot), y no a sistemas integrados como IntelliSense (Roslyn) en Visual Studio o Code Completion (ReSharper) en JetBrains Rider.

MENÚ PRINCIPAL

Panel del Menú de inicio

El menú principal de RESACA presenta una serie de botones con diferentes funcionalidades.

En primer lugar, los fundamentales: “Iniciar partida”, “Elegir minijuego”, “Opciones” y “Guardar y salir”, bastante autodescriptivos.

Entre estos, quizás el más destacable es el Menú de opciones, que presenta una serie de configuraciones básicas para el jugador (pantalla completa, volumen de la música y volumen de los efectos), así como un *toggle* para activar o desactivar la cinemática de inicio del juego.

Este *toggle* aparece únicamente tras haber desbloqueado la primera habitación (al completar el primer minijuego en el “Modo historia”), pues hasta ese momento se reproduce de forma automática por defecto.

Menú de opciones (izq.) y código para comprobar el estado del *toggle* (der.).

Del panel de “Elegir minijuego” también es importante destacar que ambos permanecerán bloqueados hasta ser completados por primera vez en el “Modo historia”, momento en el que se permitirá su rejugabilidad.

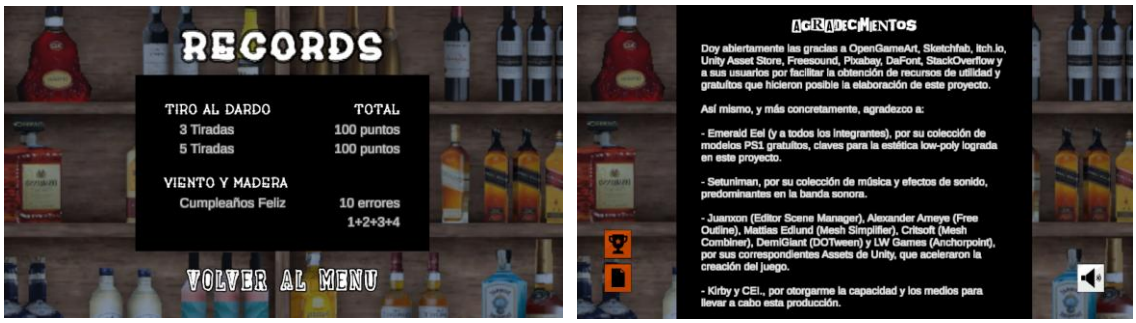
Paneles de Elegir minijuego (izq.) y Tiro al blanco (der.), con selección del número de tiradas.

En segundo lugar, tenemos los botones de las esquinas inferiores: “Récords”, “Agradecimientos” y “Mute”.

El botón de “Récords” aparece bloqueado al principio, y se desbloqueará tras haber completado los dos minijuegos disponibles en el “Modo historia”. Una vez abierto, el panel presenta los récords personales del jugador en cada uno de los minijuegos, diferenciando por puntos según el número de tiradas (en el caso de “TIRO AL BLANCO”) o por número de errores según la sección de la melodía (en el caso de “VIENTO Y MADERA”).

El botón de “Agradecimientos” muestra un panel con un texto que referencia los *assets* y agentes externos que facilitaron el desarrollo del proyecto.

El botón de “Mute” silencia por completo todos los sonidos del juego.



Paneles de Récords (izq.) y Agradecimientos (der.).

CINEMÁTICA

Al iniciar partida por primera vez, saltará una cinemática de aproximadamente treinta segundos. Esta consiste en una sucesión de cuatro planos animados con el paquete *Cinemachine* y en el uso de corrutinas para encadenarlos.

```
Frequently called   1 usage   Mario Villanueva Torres
private IEnumerator Animacion()
{
    ResetearCamaras();
    _CamaraAnimacion1.Priority = 10;
    yield return StartCoroutine( routine: Plano1());
    ResetearCamaras();
    _CamaraAnimacion2.Priority = 10;
    yield return StartCoroutine( routine: Plano2());
    ResetearCamaras();
    _CamaraAnimacion3.Priority = 10;
    yield return StartCoroutine( routine: Plano3());
    ResetearCamaras();
    _CamaraAnimacion4.Priority = 10;
    yield return StartCoroutine( routine: Plano4());
    GestorJuego.CargarEscena( id: 2);
}
```

```
Frequently called   1 usage   Mario Villanueva Torres
private IEnumerator PostprocesadoFundidos()
{
    _ColorEscena.postExposure.value = -10f;
    float temporizador = 0f;
    while (temporizador < _DuracionFade)
    {
        temporizador += Time.deltaTime;
        float t = temporizador / _DuracionFade;
        _ColorEscena.postExposure.value = Mathf.Lerp(-10,0,t);
        yield return null;
    }
    yield return new WaitForSeconds(20f);
    temporizador = 0f;
    while (temporizador < _DuracionFade)
    {
        temporizador += Time.deltaTime;
        float t = temporizador / _DuracionFade;
        _ColorEscena.postExposure.value = Mathf.Lerp(0,-10,t);
        yield return null;
    }
    _ColorEscena.postExposure.value = -10f;
    GestorSonido.Instancia.FadeOut();
}
```

Código de la corrutina que gestiona la animación (izq.) y de la que gestiona los fundidos a negro inicial y final (der.)

El objetivo de esta cinemática es presentar brevemente el escenario por el que se moverá el jugador, además de plasmar la estética y la temática tratadas en el videojuego jugando con el alto nivel de postprocesado aplicado a la imagen.

Las cámaras no recorren la misma escena con la que interactuará el jugador, sino una copia exacta sin los *Scripts* ni los elementos de la interfaz que la componen.

Fotogramas de la cinemática.

Con el botón de “Pausa” (‘P’, ‘Esc’ o ‘Tab’) se puede saltar la animación en cualquier momento.

PARTIDA

La escena principal del juego está compuesta por un habitáculo cerrado dividido en tres secciones: la habitación del protagonista, el salón y el baño.

Cada una de las secciones subsecuentes está bloqueada hasta la finalización del minijuego disponible en la sección que la precede. De esta forma, el salón no es accesible hasta completar "TIRO AL BLANCO", y el baño no lo es hasta acabar "VIENTO Y MADERA" por primera vez.

Una última puerta bloqueada, presente en el salón, intuye la presencia de más salas por recorrer (cocina, pasillo y otras habitaciones). Sin embargo, debido a las limitaciones encontradas en el desarrollo de RESACA, su funcionalidad no está terminada por el momento.

El movimiento del jugador se limita a andar y correr. En el código también se puede encontrar, sin pulir del todo, una mecánica de agacharse. Aunque finalmente no está introducida en la versión DEMO, este sistema se plantea para una potencial versión futura del desarrollo.

En el escenario están presentes más de 100 *assets* decorativos y 15 interactuables. La interacción está lograda a partir de rayos que se emiten desde la cámara del jugador hacia los elementos, utilizando interfaces para diferenciar entre los elementos interactuables y los que no. Además, se diferencia entre tres tipos de interacción:

```
Frequently called   1 usage   Mario Villanueva Torres
public void Interactuar()
{
    Ray rayo = new( origin: _TransformCamara.position,  direction: _TransformCamara.forward);
    if(Physics.Raycast(rayo,out RaycastHit datosInteraccion, _RangoDeInteraccion))
    {
        // Si es interactuable
        if(datosInteraccion.transform.TryGetComponent(out IInteractuable interactuable))
        {
            interactuable.AlInteractuar();
        }
    }
}
```

Ejemplo de *script* para "Interactuar" ('E') con un objeto.

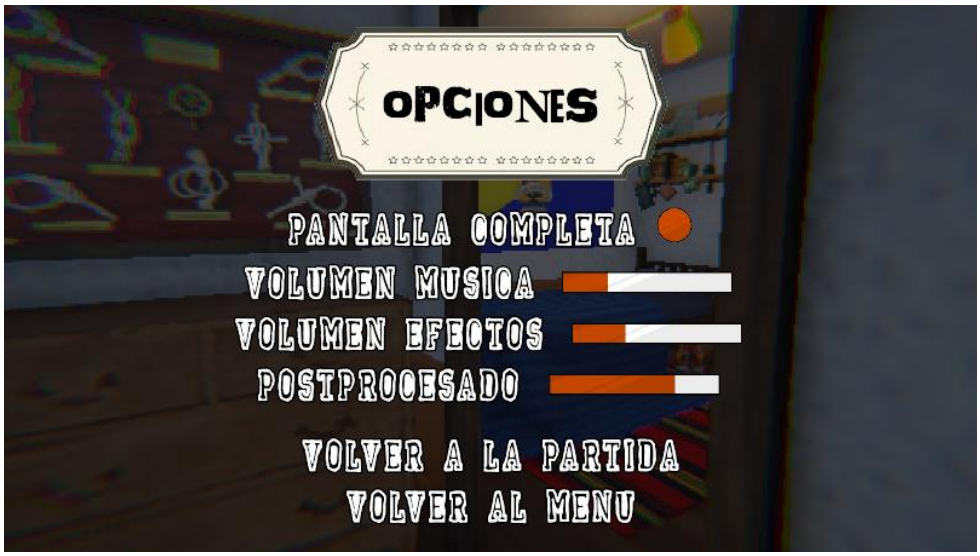
- "Ver info": automática al apuntar hacia un objeto interactuable. El contorno del *asset* se perfila de blanco (*Free Outline* de Alexander Ameye) y aparece un texto informativo con el nombre del objeto y el botón con el que interactuar en la esquina superior izquierda.
- "Interactuar": al pulsar el botón indicado ('E'), aparece un texto en formato de subtítulos que aporta información sobre el objeto. Los textos disponibles forman parte de un *array* de *strings* propio de cada objeto interactuable, con frases predeterminadas que se devuelven de manera aleatoria (algunas frases están repetidas para aumentar su ratio de aparición).

- “Jugar”: al pulsar el botón indicado (‘F’), el jugador entrará en el correspondiente minijuego al que esté apuntando: “TIRO AL BLANCO”, en la diana; o “VIENTO Y MADERA”, en la guitarra.

Fotograma de la interfaz al “Interactuar” (‘E’) con un objeto.

La lista de elementos interactivos incluye puertas, armarios, luces, etc.

Con el botón de “Pausa” (‘P’, ‘Esc’ o ‘Tab’) se abrirá un menú de opciones. Además de los parámetros básicos ya vistos en el menú de inicio, este panel también incluye un *slider* para controlar el volumen de postprocesado de la imagen, o incluso desactivarlo si se requiere. Esta función tiene el objetivo de mejorar la accesibilidad del juego, al permitir ajustar el nivel de ajustes que puedan resultar incómodos para el jugador.



Menú de opciones durante la partida.

Los componentes incluídos en el postprocesado son los siguientes:

Valores por defecto del perfil de volumen de postprocesado. En la cinemática inicial dichos valores están levemente intensificados.

Comparativa entre imágenes con el postprocesado al máximo (izq.) y desactivado (der.). En movimiento, efectos como la aberración cromática son mucho más notables, especialmente en las esquinas.

Una lista de todos los controles es visible en la habitación, tanto en la cinemática como durante el "Modo historia", de manera diegética como una tabla optométrica colgada de un armario.

Captura de pantalla que muestra los controles de manera diegética en un cartel de prueba ocular en la habitación ('E' - interactuar, 'F' - jugar, 'P' - pausar, 'WASD' - moverse y 'Shift' - correr).

MINIJUEGOS

La DEMO jugable de RESACA presenta dos minijuegos disponibles: “TIRO AL BLANCO” y “VIENTO Y MADERA”.

Cada uno está construido de tal manera que se facilita su escalabilidad en el futuro, característica que explicaré en detalle en sus respectivas descripciones.

La estructura de cada minijuego está regida por un *enum* que establece la fase actual correspondiente, y por una variable concreta de la que dependen el resto de funciones.

“TIRO AL BLANCO”

El primer minijuego consiste en lanzar dardos con un sistema de barras de carga hacia una diana, que calculará la puntuación en base a su posición final. El objetivo es alcanzar el mayor número de puntos posible.

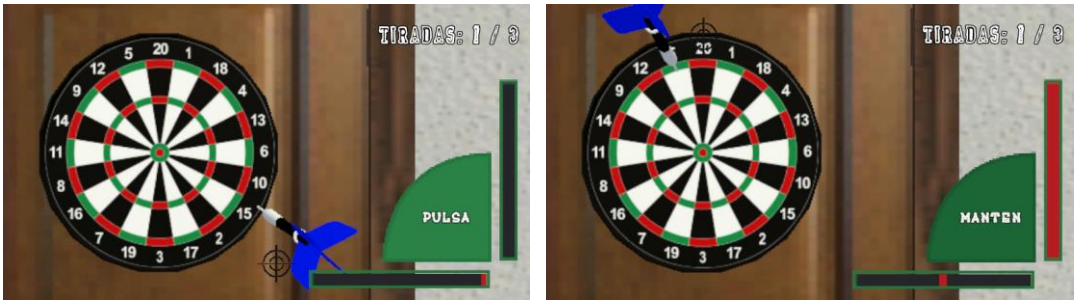
Se divide en 6 fases:

- Inicio: pantalla principal del minijuego, permite seleccionar entre los botones “Jugar”, “Cómo jugar” y “Salir”.
- Tutorial: pantalla con un breve texto que explica el funcionamiento del minijuego, accesible desde “Cómo jugar”.
- Carga Horizontal: primera fase del juego. Una barra horizontal muestra la posición izquierda-derecha a la que irá dirigido el dardo. Un *sprite* de una mira representa visualmente esa posición en la diana.
- Carga Vertical: segunda fase del juego. Una barra vertical muestra la “fuerza” del tiro (posición arriba-abajo). La mira también representa visualmente esta posición en la diana.
- Animación: con un par de cambios de plano con *Cinemachine*, se muestra cómo el dardo lanzado avanza hasta su posición correspondiente en la diana y se clava en la misma. Una vez finalizada la animación, se repite en bucle el proceso desde Carga Horizontal hasta finalizar el número total de tiradas.
- Final: se calculan los resultados en base a la posición de los dardos en la diana. La pantalla muestra los resultados de la partida actual y el récord personal del minijuego. También permite seleccionar entre “Jugar otra vez”, “Guardar nuevo récord” y “Salir”.

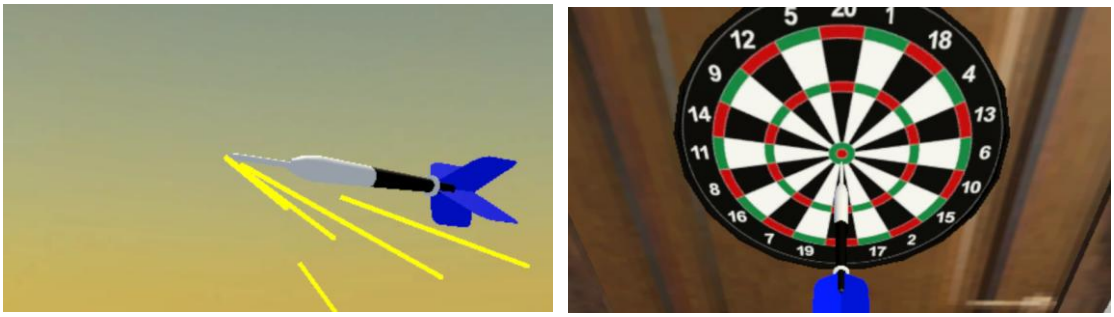
La variable fundamental de “TIRO AL BLANCO” es el número de tiradas máximas. En el “Modo historia”, este será 3 por defecto, pero desde el menú principal se podrá elegir entre 3 o 5 tiradas. Tanto el bucle de tiradas como el cálculo de resultados, el serializado de datos de guardado, los récords personales y el correcto funcionamiento del minijuego al completo están configurados para

adaptarse a esta cifra, por lo que no supondría un problema modificarla. Sin embargo, está limitado a esas dos opciones para evitar que se sienta repetitivo.

Pantallas de Inicio (izq.) y Tutorial (der.).



Pantallas de Carga Horizontal (izq.) y Carga Vertical (der.). La interfaz muestra el número de la tirada actual, la mira indicando la dirección que seguirá el dardo y un dardo tridimensional animado que se orienta hacia la diana según la Carga.



Fotogramas de la animación del dardo, saliendo disparado (izq.) y acercándose a la diana (der.). En el primer plano, un sistema de partículas y un *dolly zoom* dan efecto de velocidad al dardo.

Una característica clave de "TIRO AL BLANCO" es el cálculo de resultados. Como se puede ver en el código de la imagen inferior, la puntuación depende de la distancia del dardo clavado al centro de la diana y de ciertos multiplicadores que imitan el funcionamiento de una diana real (50 puntos en el círculo central, 25 en la circunferencia que lo rodea, x3 en la segunda circunferencia y x2 en la tercera y más exterior).

La principal complicación de este sistema, además de calcular las medidas en el espacio de trabajo tridimensional de *Unity*, fue el uso de fórmulas ajenas a mis competencias matemáticas:

- *Mathf.Atan2* para calcular el ángulo del dardo respecto al centro en radianes.

- *Mathf.Rad2Deg* para convertir el valor de radianes a grados.
- *Mathf.FloorToInt* para redondear el resultado de un cálculo hacia abajo y devolver el resultado como *int*, lo que me proporciona el sector de la diana en el que se sitúa el dardo clavado y, por consiguiente, los puntos obtenidos sin el multiplicador (sectores de 18º previamente calculados y dispuestos según el orden reconocido en una diana estándar: { 20, 1, 18, 4, 13, 6, 10, 15, 2, 17, 3, 19, 7, 16, 8, 11, 14, 9, 12, 5 }).

```
Frequently called   1 usage   Mario Villanueva Torres
public int CalcularPuntosTirada(int indiceTirada)
{
    if (indiceTirada < 0 || indiceTirada >= _GestorJuegoDiana.TiradasMaximas)
    {
        Debug.LogWarning($"Índice inválido: {indiceTirada}");
        return 0;
    }

    Vector3 posicionDardo = _AnimacionDardo.DardosClavados[indiceTirada].position;
    Vector3 direccionDardo = posicionDardo - _CentroDiana.position;
    _Radio = direccionDardo.magnitude;

    if (_Radio > _RadioFinDoble) return 0;

    if (_Radio < _RadioCentroInterior) return 50;
    if (_Radio < _RadioCentroExterior) return 25;

    // Calcular el ángulo en radianes (Atan2) y pasarlo a grados (Rad2Deg), de -180° a +180°
    _Angulo = Mathf.Atan2( y: direccionDardo.x,  x: direccionDardo.y) * Mathf.Rad2Deg;
    print( message: $"Angulo {indiceTirada}: { _Angulo}");
    if (_Angulo < 0f) _Angulo += 360f; // Convierte a 0°-360° para utilizar valores positivos

    // Dividir la diana en 20 sectores
    float anguloSector = 360f / 20f; // 18° por sector
    float medioSector = anguloSector / 2f; // 9°
    float anguloAjustado = (_Angulo + medioSector) % 360f;
    _IndiceSector = Mathf.FloorToInt( f: anguloAjustado / anguloSector) % 20;
    // Divididmos el ángulo ajustado por los 18° que mide cada sector
    // Mathf.FloorToInt redondea el resultado hacia abajo y lo devuelve como int (para obtener el n° del sector)
    // %20 aplica módulo 20 (devuelve el resto de dividir entre 20), asegurando indice válido en un array de 20 elementos

    _PuntosBase = _Sectores[_IndiceSector];

    _Multiplicador = 1;
    if (_Radio > _RadioInicioTriple && _Radio < _RadioFinTriple)
    {
        _Multiplicador = 3;
    }
    else if (_Radio > _RadioInicioDoble && _Radio < _RadioFinDoble)
    {
        _Multiplicador = 2;
    }
    Debug.Log($"Radio: {_Radio:F3}, Multi: {_Multiplicador}");

    return _PuntosBase * _Multiplicador;
}
```

Script que calcula la puntuación en cada tirada.

“VIENTO Y MADERA”

El segundo minijuego consiste en repetir la melodía previamente reproducida, al estilo “Simón dice” de otros juegos populares. El objetivo es pulsar las notas en orden y con la duración adecuada para cometer el menor número de errores posible.

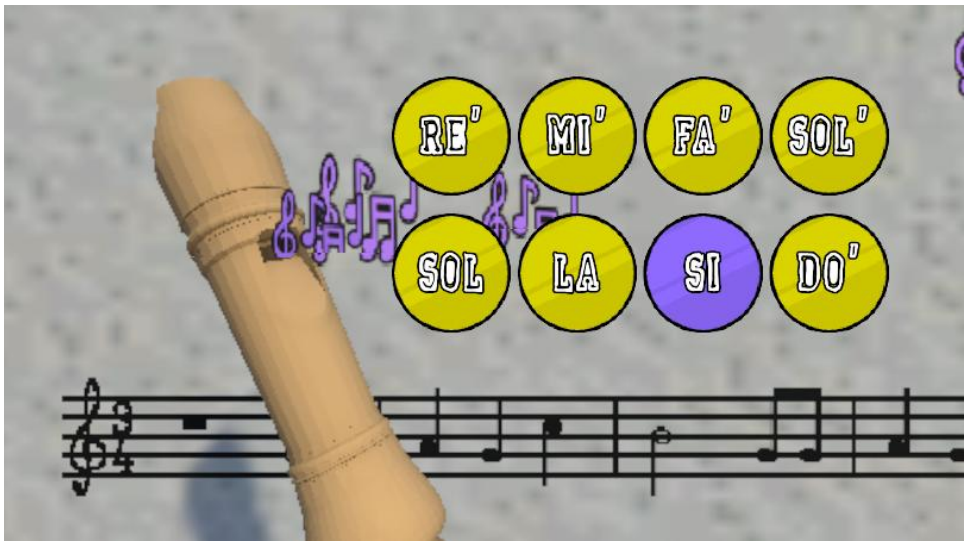
Se divide en 6 fases:

- Inicio: de nuevo, permite seleccionar entre “Jugar”, “Cómo jugar” y “Salir”.
- Tutorial: de nuevo, breve texto que explica el funcionamiento del juego.
- Melodía: primera fase del juego. Se reproduce una sección de la melodía correspondiente y se muestra visualmente la partitura.
- Jugador: segunda fase del juego. El jugador debe reproducir las notas en el mismo orden y con la misma duración que se mostró previamente en Melodía.
- Error: breve fase que se muestra únicamente si el jugador falla una nota. Un input visual muestra la nota equivocada y devuelve el bucle a Melodía para repetir la sección. En caso de no haber fallado ninguna nota, se pasa a la siguiente sección y se repite el bucle.
- Final: se calculan los resultados en base al número de errores cometidos durante las diferentes secciones. La pantalla muestra los resultados de la partida actual y el récord personal. De nuevo, permite seleccionar entre “Jugar otra vez”, “Guardar nuevo récord” y “Salir”.

La variable fundamental de este minijuego es más compleja que la anterior, pues se trata de una “lista de listas”. Más concretamente, una lista ‘B’ de notas musicales y duraciones vinculadas (que conforma un recuento de las notas de esa sección) dentro de una lista ‘A’ repleta de listas ‘B’ (que conforma un recuento de las secciones, lo que vendría a ser la melodía completa). La escalabilidad de este minijuego depende de la creación de nuevas listas ‘A’, indicando cuántas secciones la componen y qué notas componen cada sección.

El total de notas disponible se encuentra en otra lista oculta, por lo que la carencia de diferentes melodías en la DEMO de RESACA se debe únicamente a limitaciones en el tiempo de desarrollo. Potencialmente, se podría añadir cualquier melodía que incluya las notas de las 2 octavas centrales (de Do3 a Si4), preferiblemente si utiliza menos de 8 notas diferentes, y con duración de corchea, negra o blanca (duraciones representadas como 0.5, 1 o 2 para facilitar su implementación).

Pantallas de Inicio (izq.) y Tutorial (der.).



Pantalla del juego. La nota pulsada se "carga" de morado hasta completar su duración. Un sistema de partículas y el sistema de animaciones *DOTweens* (de DemiGiant) se muestran visualmente en la flauta mientras se reproducen las notas. La partitura de la melodía se desplaza por el fondo de la imagen.

Los sistemas más destacados de este minijuego son el sistema de comprobación de errores y la aplicación de competencias musicales al proyecto.

Respecto a la primera, se considera error si:

- No se pulsa ninguna nota durante 4 segundos.
- No se pulsa la nota correspondiente en el orden de la sección.
- La pulsación no dura lo que debe.

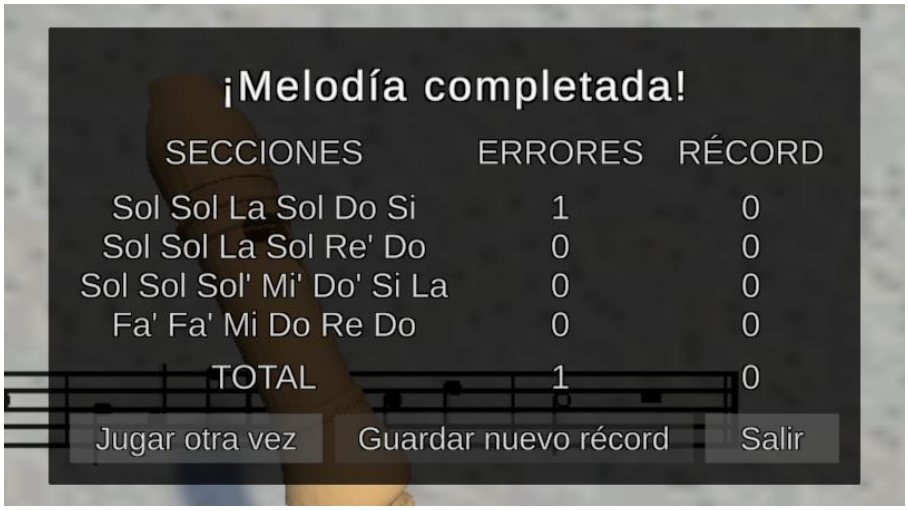
Respecto a la segunda, fueron necesarios conocimientos sobre lectura y representación de partituras, claves y escalas para adaptar adecuadamente todos estos conceptos en la interfaz del proyecto. La decisión de incluir la melodía "Cumpleaños feliz" no se debe exclusivamente a su reconocimiento global, sino que tiene el objetivo de ser visualmente sencilla para el jugador casual.

SISTEMA DE GUARDADO

El sistema de guardado consiste en la serialización de datos en un archivo JSON, combinado con el uso del sistema *PlayerPrefs* para acelerar procesos.

El guardado en RESACA no es manual, sino automático. La partida se carga al abrir el juego y se guarda al cerrarlo. Además, también se guarda en los cambios de escena para reflejar los cambios en las diferentes pantallas (por ejemplo: mostrar los récords de los minijuegos; recordar la posición del personaje al cambiar de escena; o mantener el estado de los objetos interactivos, como puertas abiertas o luces encendidas).

Otro uso del sistema de guardado es en la pantalla de resultados de los minijuegos, donde el botón "Guardar nuevo récord" se muestra desactivado si la puntuación de la partida actual es peor que la almacenada en el archivo de guardado.



Panel de resultados de "VIENTO Y MADERA". No se puede guardar un récord porque se ha cometido 1 error, mientras que el récord guardado es el de una partida perfecta (0 errores totales).

CONFLICTOS

El desarrollo de RESACA se vio obstaculizado constantemente por diversos problemas, ajenos y propios.

En primer lugar, la mayor limitación presente hasta la entrega final fue el tiempo disponible. El proyecto comenzó de manera menos ambiciosa de lo que finalizó, por lo que fue imposible abarcar todas las ideas planteadas a lo largo del proceso.

Ante la falta de competencias en modelado 3D me vi relegado a una continua búsqueda de *assets* gratuitos que mantuviesen cierta coherencia temática con el proyecto. Si bien la decisión de seguir una dirección artística *retro* fue mayoritariamente estética, también facilitaba la búsqueda al limitar los resultados a modelos más aparentemente sencillos (*low poly*).

La búsqueda de material de audio y su posterior edición y nivelado también ocupó una parte importante del tiempo total de desarrollo.

Otro incidente que destacar es el *crasheo* constante del programa *Visual Studio* durante la programación del videojuego, debido a lo que acabé asumiendo que se trataba de una sobrecarga de memoria. Este problema provocaba el bloqueo completo del ordenador cada pocos minutos de trabajo, forzando un reinicio de la máquina y ralentizando enormemente el proceso. Fue solucionado con bastante posterioridad cambiando el programa de código a *JetBrains Rider*, implicando la consecuente (aunque sencilla) adaptación al nuevo *software*.

Por último, la mayor limitación en el desarrollo de RESACA fui yo mismo. Si bien mis conocimientos al comienzo de la producción eran muy limitados, alejado de la plena comprensión de conceptos como “instancia” o “clase abstracta”, o de funciones ya mencionadas como *Mathf.Atan2* o *Mathf.FloorToInt*, siento cierta satisfacción tras la entrega del proyecto por sentir que mi entendimiento sobre estas nociones ha mejorado notablemente. De igual manera, la necesidad de documentarme acerca de los minijuegos para exponer correctamente las ideas tratadas (puntuación de los dardos, partituras musicales), aunque ya partía de una base mucho más amplia, resultó en último término gratificante.

En definitiva, a pesar de las complicaciones encontradas, considero que la realización de este proyecto constituyó una experiencia muy enriquecedora que elevó mi interés por la programación de videojuegos y su desarrollo en general.